OficioCR

Documento de planificación técnica y organización del proyecto

Requisitos · Arquitectura · Base de datos · API · Integraciones · Reparto de trabajo · Cronograma

1. Objetivo del documento

Este documento reúne la propuesta de organización técnica para OficioCR, una plataforma que conecta a clientes con trabajadores que ofrecen servicios para el hogar. Se toma como referencia el documento ERS del proyecto y se complementa con propuestas prácticas para poder implementarlo en un periodo de 6 semanas.

Importante: los requisitos RF-01 a RF-11 y RNF-01 a RNF-12 se presentan como requisitos recogidos en el ERS. Los requisitos RF-12 a RF-14, RNF-13, el diseño de tablas, los endpoints y el cronograma son propuestas de planificación para concretar la implementación; deben validarse con el equipo y el profesorado.

2. Resumen del proyecto

OficioCR permitirá a los clientes buscar profesionales por categoría y ubicación, solicitar servicios, comunicarse con los trabajadores y valorar los trabajos completados. Los trabajadores podrán publicar sus servicios y gestionar las solicitudes recibidas. La plataforma también contempla recomendaciones mediante un asistente de IA, pagos a través de una pasarela y funciones administrativas.

Actores principales

- Cliente: busca servicios, envía solicitudes, conversa con el trabajador, realiza el pago cuando corresponda y deja una valoración.
- Trabajador: publica servicios, define precio y zona de cobertura, recibe solicitudes y actualiza su estado.
- Administrador: supervisa el funcionamiento y gestiona incidencias o cuentas según los permisos definidos.
- Asistente de IA: interpreta la descripción de una necesidad y ayuda a encontrar categorías o servicios relevantes.

3. Requisitos funcionales del ERS

RF-01. Registro de usuarios — Permitir que una persona cree una cuenta con los datos requeridos por el sistema.

RF-02. Inicio de sesión — Permitir el acceso a usuarios registrados y controlar el acceso según su rol.

RF-03. Publicación de servicios — Permitir al trabajador publicar servicios con nombre, categoría, descripción, precio y zona de cobertura.

RF-04. Búsqueda de trabajadores — Permitir al cliente buscar por categoría, ubicación y filtros disponibles.

RF-05. Solicitud de servicio — Permitir al cliente enviar al trabajador una solicitud con descripción, dirección y fecha deseada; la solicitud comienza pendiente.

RF-06. Recomendaciones mediante IA — Interpretar una descripción en lenguaje natural y la ubicación para recomendar servicios o trabajadores pertinentes.

RF-07. Valoraciones — Permitir una puntuación de 1 a 5 estrellas y un comentario únicamente para trabajos completados, con una valoración por contrato.

RF-08. Pago de servicios — Integrar una pasarela de pago, registrar el estado de la transacción y generar un comprobante.

RF-09. Historial de pagos — Permitir consultar el historial de pagos asociados al usuario.

RF-10. Mensajería directa — Permitir mensajes entre cliente y trabajador y conservarlos en orden cronológico.

RF-11. Notificaciones de mensajes — Notificar mensajes nuevos y mantener el recuento o estado de lectura.

4. Requisitos no funcionales del ERS

RNF-01. Rendimiento — El 95 % de las búsquedas o consultas debe responder en 3 segundos o menos con hasta 50 usuarios concurrentes.

RNF-02. Seguridad — Almacenar contraseñas mediante hash, usar HTTPS y restringir el acceso según los roles.

RNF-03. Disponibilidad — Objetivo de disponibilidad mensual del 99 %, excluyendo mantenimiento planificado.

RNF-04. Usabilidad — Al menos el 80 % de los usuarios debe poder completar tareas básicas sin ayuda.

RNF-05. Compatibilidad — Compatibilidad con las dos versiones estables más recientes de Chrome, Edge y Firefox.

RNF-06. Diseño adaptable — Interfaz usable en anchos aproximados de 360, 768 y 1366 píxeles.

RNF-07. Integridad de datos — Aplicar restricciones relacionales y validaciones para mantener la consistencia.

RNF-08. Seguridad de pagos — Evitar transacciones duplicadas y no almacenar el CVV ni datos de tarjeta que no deban conservarse.

RNF-09. Mantenibilidad — Código modular, uso de GitHub y documentación del proyecto.

RNF-10. Copias de seguridad — Realizar copias de seguridad diarias de la base de datos y probar la restauración.

RNF-11. Privacidad de conversaciones — Restringir el acceso a las conversaciones a sus participantes autorizados.

RNF-12. Gestión de errores — Informar de errores de forma controlada y facilitar la recuperación cuando sea posible.

5. Requisitos adicionales propuestos

Estas ampliaciones no deben confundirse con los requisitos originales del ERS; se proponen para cerrar detalles necesarios de implementación.

RF-12. Ciclo de vida de la solicitud — Definir las transiciones permitidas: PENDIENTE, ACEPTADA, RECHAZADA, COMPLETADA y CANCELADA. Validar quién puede ejecutar cada cambio y bajo qué condiciones.

RF-13. Administración básica — Permitir al administrador consultar usuarios, desactivar cuentas cuando corresponda y revisar informes sencillos.

RF-14. API documentada — Definir endpoints REST, formatos de petición/respuesta, códigos de error y permisos; mantener la documentación disponible mediante OpenAPI/Swagger.

RNF-13. Integraciones externas seguras — Mantener las claves de IA y pagos únicamente en el backend, configurar tiempos de espera y tratamiento de errores, y ofrecer una alternativa manual si la IA no está disponible.

6. Arquitectura técnica propuesta

La propuesta utiliza una arquitectura cliente-servidor sencilla, adecuada para un proyecto académico con un plazo limitado.

- Frontend: Angular para las pantallas, formularios, búsqueda, perfiles, solicitudes y mensajería.
- Backend: ASP.NET Core Web API para las reglas de negocio, autenticación, autorización, validación y comunicación con servicios externos.
- Base de datos: SQL Server, de acuerdo con la tecnología indicada en el ERS.
- Acceso a datos: Entity Framework Core y migraciones para versionar los cambios del esquema.
- Autenticación: ASP.NET Core Identity o una solución equivalente basada en tokens; proteger endpoints según rol y propiedad de los recursos.
- Documentación y pruebas de API: Swagger/OpenAPI.
- Control de versiones: GitHub, con ramas y pull requests acordados por el equipo.

Flujo general: Angular envía una petición HTTP a la API de ASP.NET Core; el backend valida identidad, permisos y datos, ejecuta la lógica de negocio y consulta o modifica SQL Server. Para IA y pagos, el backend se comunica con el proveedor externo, sin exponer secretos en el navegador.

7. Modelo de datos inicial

El siguiente modelo es una propuesta inicial derivada de los módulos del ERS. Los nombres y cardinalidades pueden ajustarse al diseño definitivo. Se recomienda definir las claves foráneas, índices y restricciones antes de desarrollar las pantallas.

Entidad	Campos principales propuestos
USUARIO	Id (PK), Nombre, Correo (único), PasswordHash, Telefono, Rol, Activo
CATEGORIA	Id (PK), Nombre (único)
SERVICIO	Id (PK), TrabajadorId (FK → USUARIO), CategoriaId (FK → CATEGORIA), Nombre, Descripcion, Precio, Zona, Activo
SOLICITUD	Id (PK), ClienteId (FK → USUARIO), ServicioId (FK → SERVICIO), Descripcion, Direccion, FechaSolicitada, Estado
CALIFICACION	Id (PK), SolicitudId (FK → SOLICITUD, único), Puntuacion, Comentario
PAGO	Id (PK), SolicitudId (FK → SOLICITUD), Monto, Estado, ReferenciaExterna, EsPrueba
CONVERSACION	Id (PK), CreadaEn
CONVERSACION_PARTICIPANTE	ConversacionId (PK/FK), UsuarioId (PK/FK)
MENSAJE	Id (PK), ConversacionId (FK → CONVERSACION), RemitenteId (FK → USUARIO), Contenido, EnviadoEn
NOTIFICACION	Id (PK), UsuarioId (FK → USUARIO), Tipo, Contenido, Leida
RECOMENDACION_IA (opcional)	Id (PK), UsuarioId (FK → USUARIO), Consulta,

	CategoriaDetectada, CreadaEn

Relaciones principales

- Un usuario con rol trabajador puede publicar varios servicios; cada servicio pertenece a una categoría.
- Un cliente puede crear varias solicitudes y cada solicitud se relaciona con un servicio publicado.
- Una solicitud puede tener cero o una valoración, que solo se permite al completarse el trabajo.
- Una solicitud puede tener un registro de pago o varios intentos, según la pasarela y las necesidades del proyecto. Para el MVP, simplificar a un pago principal si es suficiente.
- Una conversación tiene participantes mediante CONVERSACION_PARTICIPANTE y contiene mensajes ordenados cronológicamente.
- Un usuario puede recibir varias notificaciones y enviar mensajes.
- La tabla RECOMENDACION_IA es opcional: no es necesario guardar cada respuesta de IA si no existe una necesidad clara.

8. Reglas de negocio importantes

- El backend debe comprobar que el usuario que crea una solicitud tiene rol de cliente y que el servicio está activo.
- Solo el trabajador destinatario debe poder aceptar o rechazar una solicitud; el cliente podrá cancelarla únicamente bajo las condiciones que defina el equipo.
- Solo una solicitud completada puede recibir valoración y no debe aceptarse más de una valoración por solicitud.
- Un usuario solo puede leer conversaciones de las que sea participante.
- El precio y el estado del pago deben verificarse en el backend; no confiar en importes o estados enviados por Angular.
- No almacenar CVV ni secretos de pasarela. En pruebas, utilizar el modo sandbox o una simulación controlada.
- Validar las entradas del usuario, limitar los datos devueltos y no exponer detalles técnicos sensibles en mensajes de error.

9. Diseño de API REST propuesto

Los siguientes endpoints son ejemplos iniciales, no endpoints confirmados en el ERS. Deben ajustarse al contrato final y a las convenciones del equipo.

Método	Endpoint	Propósito
POST	/api/auth/register	Crear una cuenta
POST	/api/auth/login	Iniciar sesión
GET	/api/services	Consultar y filtrar servicios
POST	/api/services	Publicar un servicio (trabajador)
GET	/api/services/{id}	Consultar detalle de un servicio
POST	/api/requests	Crear una solicitud de servicio
GET	/api/requests	Consultar solicitudes propias según rol
PATCH	/api/requests/{id}/status	Cambiar estado validando permisos y transición
POST	/api/ai/recommendations	Solicitar recomendaciones a partir de una descripción y ubicación
GET	/api/conversations	Consultar conversaciones del

		usuario
POST	/api/conversations	Crear o iniciar una conversación
GET	/api/conversations/{id}/messages	Consultar mensajes si el usuario participa
POST	/api/conversations/{id}/messages	Enviar un mensaje
GET	/api/payments/history	Consultar el historial de pagos del usuario
POST	/api/payments/checkout	Iniciar pago de prueba si se implementa pasarela sandbox

Buenas prácticas para la API

- Usar DTOs para separar el contrato HTTP de las entidades de base de datos.
- Validar datos en el backend y devolver códigos HTTP coherentes (por ejemplo, 400, 401, 403, 404 y 409).
- No permitir que el cliente envíe libremente campos sensibles como el rol, el propietario de un servicio o el estado de un pago.
- Mantener documentación Swagger/OpenAPI actualizada.
- Centralizar el manejo de errores y registrar información técnica sin guardar contraseñas, tokens ni datos de pago sensibles.

10. Integración de IA

El proveedor de IA todavía no está decidido. Para reducir el riesgo del proyecto, se recomienda encapsular la integración detrás de una interfaz de backend (por ejemplo, IARecommendationService) para poder cambiar de proveedor sin modificar el resto de la aplicación.

Flujo recomendado

1. El cliente escribe qué necesita y proporciona una ubicación o zona.
2. El backend valida y envía el texto al proveedor de IA mediante una clave guardada en configuración segura.
3. La IA devuelve una categoría o descripción estructurada de la necesidad.
4. El backend busca en SQL Server servicios reales que coincidan con esa categoría y ubicación.
5. La API devuelve recomendaciones basadas en los servicios encontrados, no únicamente en texto generado por la IA.
6. Si la IA falla o no está disponible, la aplicación mantiene la búsqueda manual por categoría y ubicación.

Para el MVP no hace falta entrenar un modelo propio ni montar una base vectorial. Puede utilizarse una API externa o una implementación simulada durante el desarrollo. Antes de elegir proveedor, comparar coste, límites gratuitos, calidad en español, condiciones de privacidad, facilidad de integración y disponibilidad.

11. Pagos: solo pruebas

Como los pagos se realizarán únicamente en entorno de pruebas, se debe utilizar el modo sandbox de una pasarela o una simulación que permita demostrar el flujo sin realizar cargos reales. Debe quedar claramente indicado que el pago es de prueba.

- Registrar el importe, la solicitud asociada, el estado y una referencia de prueba.
- No guardar números completos de tarjeta ni CVV.
- Evitar pagos duplicados mediante una clave de idempotencia o una comprobación equivalente, si la solución lo permite.

- Tratar los estados de pago de forma explícita: pendiente, aprobado, rechazado o cancelado, según el diseño final.
- No marcar un pago como aprobado basándose únicamente en una respuesta del frontend.

12. Reparto de trabajo para tres integrantes

Este reparto es una propuesta para trabajar en paralelo. Cada integrante debería encargarse de su módulo de frontend, backend, persistencia y pruebas; las decisiones comunes deben acordarse antes de desarrollar.

Integrante 1 — Usuarios, autenticación y administración

- RF-01: registro de usuarios.
- RF-02: inicio de sesión y permisos por rol.
- RF-13 (propuesto): administración básica.
- Responsable principal de Usuario, roles, validación de perfiles y criterios comunes de autenticación.

Integrante 2 — Servicios, búsqueda, solicitudes y valoraciones

- RF-03: publicación de servicios.
- RF-04: búsqueda y filtros.
- RF-05: creación de solicitudes.
- RF-07: valoraciones.
- RF-12 (propuesto): estados y reglas de ciclo de vida de la solicitud.
- Responsable principal de Categoría, Servicio, Solicitud y Calificación.

Integrante 3 — IA, mensajería, notificaciones y pagos de prueba

- RF-06: recomendaciones de IA.
- RF-08 y RF-09: pago de prueba e historial.
- RF-10 y RF-11: mensajería y notificaciones.
- RF-14 (propuesto): coordinación de contratos API e integraciones externas.
- Responsable principal de Conversación, Participantes, Mensaje, Notificación y Pago.

Acuerdos compartidos antes de programar

- Acordar nombres de entidades, propiedades, DTOs, endpoints, roles y estados de solicitud.
- Definir quién crea las migraciones iniciales de SQL Server y cómo se integran los cambios posteriores.
- Mantener una rama principal estable y trabajar mediante ramas de funcionalidad y pull requests.
- Usar datos de prueba y respuestas simuladas cuando un módulo dependa de otro que aún no está listo.
- Revisar en conjunto seguridad, responsive design, accesibilidad básica, rendimiento y gestión de errores.

Si el calendario se reduce o se retrasa

Priorizar el flujo principal: registro e inicio de sesión, publicación y búsqueda de servicios, creación y gestión de solicitudes, y una demostración funcional de IA y pagos de prueba. Si el calendario se reduce, simplificar las notificaciones avanzadas, los informes administrativos secundarios y cualquier función no imprescindible para demostrar el recorrido completo.

13. Orden recomendado de implementación

7. Crear el repositorio y la solución base, y definir convenciones de nombres.

8. Diseñar el esquema inicial de SQL Server y crear las migraciones de EF Core.
9. Implementar autenticación, roles y autorización de endpoints.
10. Completar servicios, búsqueda y solicitudes: son el flujo central del producto.
11. Añadir estados de solicitud y valoraciones con reglas de negocio.
12. Integrar mensajería y notificaciones básicas.
13. Añadir recomendaciones de IA mediante una interfaz intercambiable y búsqueda real en la base de datos.
14. Implementar pago sandbox o simulación, con historial de transacciones de prueba.
15. Probar permisos, validaciones, errores, rendimiento básico y adaptación a distintos tamaños de pantalla.
16. Preparar documentación, datos de demostración y una presentación final.

14. Riesgos y decisiones pendientes

- Proveedor de IA: aún pendiente de elegir. Mantener una interfaz común y evitar depender demasiado pronto de una API concreta.
- Pasarela de pago: elegir una opción con sandbox o definir una simulación claramente identificada como prueba.
- Disponibilidad del 99 %: puede ser difícil garantizarla en un prototipo académico; documentar el objetivo y las limitaciones del entorno de despliegue.
- Notificaciones: decidir si bastan notificaciones internas y actualización periódica o si se necesita tiempo real. Para el plazo disponible, empezar por la opción más sencilla.
- Pagos y valoraciones: concretar cardinalidad, reglas de reembolso/cancelación si aplica, y cuándo una solicitud se considera completada.
- Alcance del MVP: el ERS abarca muchas funcionalidades; priorizar el recorrido central y evitar añadir funcionalidades que no se puedan probar de extremo a extremo.

15. Criterio de finalización del MVP

Se puede considerar que la versión mínima está lista para demostración cuando:

- Un usuario puede registrarse e iniciar sesión con permisos acordes a su rol.
- Un trabajador puede publicar un servicio y un cliente puede encontrarlo mediante búsqueda.
- El cliente puede crear una solicitud y el trabajador puede gestionarla mediante estados válidos.
- Una solicitud completada puede recibir una única valoración.
- Cliente y trabajador pueden intercambiar mensajes con acceso restringido a los participantes.
- La aplicación puede mostrar recomendaciones de IA o, si el proveedor no está disponible, permitir la búsqueda manual.
- El flujo de pago se puede demostrar en sandbox o mediante una simulación que no cobre dinero real.
- La API está documentada y los principales casos de éxito y error se han probado.

16. Plan de trabajo Scrum para 6 semanas

Se propone trabajar con 3 sprints de 2 semanas cada uno. Para un proyecto académico de solo 6 semanas, esta cadencia permite planificar, construir una versión funcional, integrar los módulos y reservar tiempo para pruebas. El equipo debe adaptar el plan en cada Sprint Planning según la velocidad real y los criterios de evaluación.

16.1. Roles y eventos Scrum

- Product Owner: una persona del equipo que prioriza el Product Backlog, aclara dudas sobre requisitos y valida que cada incremento aporte valor. Si el profesor actúa como cliente, conviene acordar con él las prioridades.
- Scrum Master: facilita las reuniones, ayuda a eliminar bloqueos y promueve que se respeten los acuerdos del equipo. En un equipo pequeño, este rol puede rotar.
- Developers: los tres integrantes que diseñan, implementan, prueban e integran el producto. Cada integrante puede liderar un módulo, pero el incremento es responsabilidad compartida.
- Sprint Planning: al comienzo de cada sprint, acordar el objetivo, seleccionar tareas y definir cómo se comprobará el resultado.
- Daily Scrum: reunión breve diaria de unos 10–15 minutos para coordinar el trabajo, identificar bloqueos y ajustar el plan del día.
- Sprint Review: al final de cada sprint, demostrar lo que funciona y recoger comentarios del profesor o usuarios de prueba.
- Sprint Retrospective: al final de cada sprint, decidir una o dos mejoras concretas para la forma de trabajar.
- Refinement: revisar y dividir historias del Product Backlog durante la semana para que estén listas para un próximo sprint.

16.2. Product Backlog priorizado

Prioridad propuesta para que el flujo principal funcione pronto. P0 significa imprescindible para la demostración; P1 importante si el tiempo lo permite; P2 aplazable si hay riesgo de no terminar.

Prioridad	Elemento del backlog	Requisitos relacionados
P0	Registro, inicio de sesión y roles básicos	RF-01, RF-02
P0	Publicar servicios y categorías	RF-03
P0	Buscar servicios por categoría y zona	RF-04
P0	Crear solicitudes y gestionar sus estados	RF-05, RF-12 propuesto
P0	Persistencia SQL Server y API documentada	RNF-07, RF-14 propuesto
P0	Pruebas de integración del flujo principal y control de permisos	RNF-02, RNF-12
P1	Valoración tras completar una solicitud	RF-07
P1	Recomendaciones IA con proveedor o simulación y búsqueda manual alternativa	RF-06, RNF-13 propuesto
P1	Mensajería básica entre cliente y trabajador	RF-10
P1	Pago de prueba o simulación e historial básico	RF-08, RF-09
P2	Notificaciones avanzadas o tiempo real	RF-11
P2	Informes administrativos	RF-13 propuesto

	secundarios y pulido adicional	

16.3. Sprint 1 — Semanas 1 y 2: Base técnica y flujo mínimo

Objetivo del sprint: disponer de una aplicación ejecutable con base de datos, autenticación básica y el inicio del flujo de servicios.

Historias y tareas

- Acordar alcance del MVP, criterios de aceptación, roles, estados de solicitud, DTOs y convenciones de nombres.
- Crear repositorio GitHub, estructura Angular y ASP.NET Core Web API, configuración local y Swagger/OpenAPI.
- Diseñar el esquema inicial SQL Server y configurar Entity Framework Core y migraciones.
- Implementar registro, inicio de sesión y autorización básica por rol.
- Crear categorías y CRUD básico para publicar y consultar servicios.
- Construir las primeras pantallas Angular para registro/login y listado/publicación de servicios.
- Añadir datos semilla o datos de prueba para facilitar el desarrollo y las demostraciones.

Reparto sugerido

- Integrante 1: autenticación, roles y perfil de usuario; ayuda a definir la configuración común.
- Integrante 2: entidades, categorías, publicación y consulta de servicios.
- Integrante 3: estructura inicial de conversaciones/IA/pagos según contratos acordados; prepara prototipos o mocks sin bloquear el flujo principal.
- Todos: revisión de API, migraciones, integración y pruebas al final de la semana 2.

Definición de terminado del Sprint 1

- La solución se ejecuta en los equipos del grupo siguiendo instrucciones documentadas.
- Angular se comunica con ASP.NET Core y la API puede leer/escribir en SQL Server.
- Registro e inicio de sesión funcionan en el alcance acordado y los endpoints protegidos requieren autenticación.
- Un trabajador puede crear un servicio y el listado puede mostrar los servicios almacenados.
- El código está en GitHub, integrado en la rama principal y con las pruebas básicas ejecutadas.

16.4. Sprint 2 — Semanas 3 y 4: Solicitudes y funciones de valor

Objetivo del sprint: permitir el recorrido principal de cliente y trabajador, desde buscar un servicio hasta gestionar una solicitud.

Historias y tareas

- Implementar búsqueda y filtros por categoría y zona.
- Permitir al cliente crear una solicitud con descripción, dirección y fecha deseada.
- Implementar estados PENDIENTE, ACEPTADA, RECHAZADA, COMPLETADA y CANCELADA, validando los permisos y transiciones.
- Crear pantallas para consultar solicitudes y mostrar su estado a cliente y trabajador.
- Implementar valoraciones de 1 a 5 estrellas y comentario solo cuando la solicitud esté completada, con una valoración por solicitud.
- Añadir pruebas para permisos, validaciones, solicitudes duplicadas si aplica y estados inválidos.
- Integrar un primer flujo de mensajería; si el tiempo es limitado, priorizar envío y consulta de mensajes antes de notificaciones avanzadas.
- Definir e implementar una prueba de concepto de IA: respuesta simulada o proveedor sencillo detrás de una interfaz de servicio.

Reparto sugerido

- Integrante 1: completar permisos, perfiles y apoyo a pruebas de autenticación.
- Integrante 2: liderar búsqueda, solicitudes, transiciones de estado y valoraciones.
- Integrante 3: mensajería básica y prueba de concepto de IA; definir el flujo de pago de prueba sin cargos reales.
- Todos: integrar frontend y backend de cada historia y demostrar el recorrido principal al cierre del sprint.

Definición de terminado del Sprint 2

- El cliente puede buscar un servicio y crear una solicitud persistida en SQL Server.
- El trabajador destinatario puede aceptar o rechazar la solicitud y marcarla completada cuando corresponda.
- El backend impide transiciones no permitidas y usuarios sin permisos no pueden modificar solicitudes ajenas.
- Solo una solicitud completada admite una valoración y no se puede registrar más de una.
- El flujo principal está integrado y se puede demostrar de principio a fin con datos de prueba.

16.5. Sprint 3 — Semanas 5 y 6: Integración, pruebas y entrega

Objetivo del sprint: cerrar las funcionalidades prioritarias, asegurar la estabilidad y preparar una demostración final coherente.

Historias y tareas

- Completar mensajería básica y controles para que solo los participantes puedan leer una conversación.
- Terminar la integración de IA o una simulación convincente; las recomendaciones deben apoyarse en servicios reales de SQL Server y mantener la búsqueda manual como alternativa.
- Implementar pago sandbox o simulación claramente identificada como prueba e historial básico de transacciones.
- Implementar notificaciones internas sencillas si el flujo principal ya está estable; aplazar tiempo real si amenaza la entrega.
- Completar el mínimo de administración que se haya acordado y corregir errores prioritarios.
- Ejecutar pruebas de integración y regresión, verificar roles, validaciones, manejo de errores y diseño adaptable.
- Revisar tiempos de respuesta en búsquedas, documentación Swagger, instrucciones de instalación y datos de demostración.
- Preparar guion de demo, capturas si se necesitan y documentación final; reservar los últimos días para corregir, no para empezar funciones grandes.

Reparto sugerido

- Integrante 1: cierre de autenticación, permisos, administración básica y revisión de seguridad.
- Integrante 2: cierre del flujo de servicios/solicitudes/valoraciones y pruebas funcionales.
- Integrante 3: cierre de IA, mensajería y pago de prueba, con tratamiento de errores.
- Todos: pruebas de regresión, corrección de incidencias, documentación, preparación y ensayo de la presentación.

Definición de terminado del Sprint 3

- Las funcionalidades P0 están integradas en la rama principal y se pueden ejecutar siguiendo la guía del proyecto.
- El flujo principal se ha probado con cuentas de cliente y trabajador distintas.
- No quedan errores críticos conocidos que impidan registro, búsqueda, solicitud o gestión de estados.
- IA y pagos se demuestran mediante integración real o simulación claramente indicada; no se realizan cargos reales.
- La API está documentada y existe una guía breve para ejecutar el proyecto y preparar la demo.

16.6. Ceremonias y ritmo de trabajo

- Día 1 de cada sprint: Sprint Planning de 60–90 minutos para definir objetivo, historias y tareas.
- Cada día laborable: Daily Scrum de 10–15 minutos; revisar qué se terminó, qué sigue y qué bloquea.
- Durante el sprint: refinement de 30 minutos una o dos veces por semana para aclarar historias del siguiente sprint.
- Último día de cada sprint: Sprint Review de 30–45 minutos con una demo funcional y feedback.
- Después de la Review: Sprint Retrospective de 30 minutos y un compromiso de mejora para el siguiente sprint.

16.7. Reglas de alcance para proteger las seis semanas

- No iniciar una funcionalidad P1 o P2 mientras haya historias P0 bloqueadas o sin integrar.
- No dejar la integración para el último sprint: integrar código en pequeños incrementos durante todas las semanas.
- Elegir pronto entre IA real y simulación; si la decisión se retrasa, usar una interfaz y un mock para continuar.
- Para pagos, limitarse al sandbox o simulación; no invertir tiempo en un flujo de producción real.
- Si hay retrasos, aplazar primero notificaciones en tiempo real, informes secundarios y refinamientos visuales no esenciales.
- Cada historia debe tener un criterio de aceptación comprobable y estar integrada antes de considerarse terminada.

17. Ajustes del plan original

El cronograma inicial de 8–12 semanas queda sustituido por el plan de 6 semanas descrito en esta sección. Para que el alcance sea realista, el equipo debe priorizar el recorrido completo registro → publicación/búsqueda → solicitud → gestión del estado. Mensajería, IA y pagos se desarrollan en versión mínima o simulada, y las notificaciones avanzadas quedan como trabajo aplazable. La estimación de trabajo debe revisarse en cada Sprint Planning.